

MoveInsight: Herramienta para el análisis de patrones de movimiento y prevención de lesiones

Anexo VI. Manual del programador

Trabajo de Fin de Grado

Grado en Ingeniería Informática



**VNiVERSiDAD
D SALAMANCA**

Junio de 2026

Autor

Diego Gómez Terradillos

Tutora

Dña. Carolina Zato Domínguez

Índice de Contenidos

Índice de Contenidos	i
Índice de Figuras	ii
1 Introducción	1
1.1 Visión de la arquitectura y organización del repositorio	1
2 Backend	5
2.1 Arranque y configuración	5
2.2 Capa de datos: ORM y base de datos	6
2.3 DTOs y validación	7
2.4 Autenticación y seguridad	8
2.5 Routers por dominio	8
2.6 Servicios de lógica de negocio	10
2.7 Pipeline de IA en inferencia	11
2.8 Contrato de la API REST y documentación OpenAPI	13
3 Frontend	16
3.1 Configuración Gradle	16
3.2 Punto de entrada	17
3.3 Capa core	17
3.4 Capa data	19
3.5 Capa domain	19
3.6 Capa presentation	20
3.7 Navegación y tema	22
4 Entrenamiento	23
4.1 Preparación de datos: etiquetado, <i>features</i> y dataset	24
4.2 Entrenamiento de modelos	25
4.3 Evaluación, versionado y producción	25
Glosario de términos	27

Índice de Figuras

Figura 1. Raiz del repositorio	2
Figura 2. Carpeta Backend	3
Figura 3. Carpeta frontend/app	3
Figura 4. Carpeta backend/app	5
Figura 5. Carpeta backend/app/routers	8
Figura 6. Carpeta backend/app/services	10
Figura 7. Carpeta backend/processing.....	11
Figura 8. Carpeta backend/modelos	13
Figura 9. Documentación de la API /docs.....	14
Figura 10. Documentación de la API /redoc	15
Figura 11. Documentación de la API /openapi.json.....	15
Figura 12. Carpeta frontend/src/com.moveinsight	16
Figura 13. Carpeta frontend/src/gradle	16
Figura 14. Carpeta frontend/src/com.moveinsight/core	17
Figura 15. Carpeta frontend/src/com.moveinsight/data	19
Figura 16. Carpeta frontend/src/com.moveinsight/data	19
Figura 17. Carpeta frontend/src/com.moveinsight/presentation	20
Figura 18. Carpeta frontend/src/com.moveinsight/presentation/navigation	22
Figura 19. Carpeta backend/entrenamiento	23

1 Introducción

Este anexo tiene como objetivo proporcionar una descripción detallada de la arquitectura interna, la organización del código fuente, los módulos que componen la aplicación y las decisiones de diseño más relevantes adoptadas durante el desarrollo, para así hacer posible el desarrollo por otras personas.

El lector de este manual se asume que dispone de conocimientos en desarrollo de software, en particular en los ecosistemas Python/FastAPI y Android/Kotlin, así como nociones de contenedorización con Docker y de redes neuronales aplicadas a señales temporales. No constituye un tutorial a estas tecnologías, sino una referencia interna que explica cómo está construido MoveInsight y por qué cada parte del sistema está organizada de la manera en que lo está.

El alcance del manual abarca los tres bloques principales del proyecto:

- **Backend:** la API REST construida con FastAPI, la capa de acceso a datos sobre MariaDB y el pipeline de inteligencia artificial que procesa los vídeos de sentadillas.
- **Frontend:** la aplicación Android desarrollada en Kotlin con Jetpack Compose, organizada según los principios de Clean Architecture y el patrón MVVM.
- **Entrenamiento:** los scripts y conjuntos de datos utilizados para entrenar y versionar los modelos de segmentación de fases (BiLSTM) y clasificación de KPIs (XGBoost).

Quedan fuera del alcance de este anexo la configuración del servidor de producción, los certificados TLS y la orquestación Docker, aspectos tratados íntegramente en el Anexo II.

1.1 Visión de la arquitectura y organización del repositorio

MoveInsight sigue una arquitectura cliente-servidor clásica dividida en dos piezas independientes que se comunican exclusivamente a través de una API REST.

El backend es una aplicación Python construida sobre FastAPI que actúa simultáneamente como servidor de la API y como motor de procesamiento de vídeo. Recibe las peticiones HTTP del cliente, gestiona la autenticación de usuarios mediante tokens JWT, persiste los datos en una base de datos relacional MariaDB a través del ORM SQLAlchemy y, cuando el usuario sube un vídeo de sentadilla, lanza un pipeline de IA en segundo plano. Dicho pipeline extrae la pose humana fotograma a fotograma con

MediaPipe BlazePose, calcula características biomecánicas, segmenta las repeticiones con un modelo BiLSTM y clasifica la calidad de cada repetición en cinco KPIs (profundidad, torso, valgo, simetría y ritmo) mediante modelos XGBoost independientes. El resultado queda almacenado en la base de datos y, adicionalmente, se generan un vídeo con el esqueleto superpuesto y una imagen de informe en PNG, accesibles posteriormente por el cliente.

El frontend es una aplicación Android nativa desarrollada en Kotlin con Jetpack Compose. Implementa Clean Architecture en cuatro capas (core, data, domain y presentation) y el patrón MVVM con StateFlow para la gestión reactiva del estado. Se comunica con el backend mediante Retrofit y gestiona de forma automática el ciclo de vida de los tokens JWT, incluyendo el refresco transparente ante respuestas 401. La lógica de notificaciones periódicas (check-ins de dolor y resúmenes semanales) se delega en WorkManager.

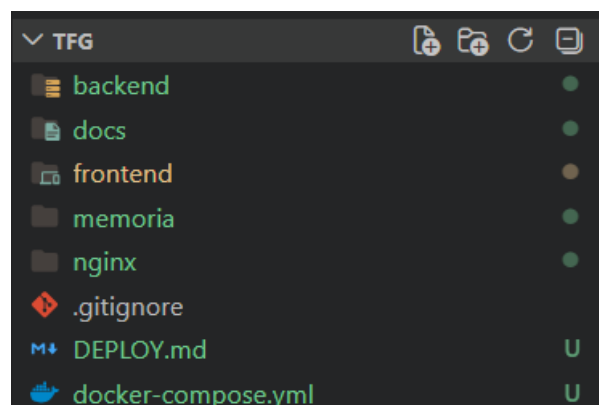


Figura 1. Raiz del repositorio

Dentro de backend/, el código se divide en tres grandes bloques. La carpeta app/ contiene la API propiamente dicha: el punto de entrada (main.py), la configuración centralizada (config.py), los modelos ORM (models.py), los esquemas Pydantic (schemas.py), la autenticación (auth.py), los routers por dominio (routers/) y los servicios de lógica de negocio (services/). La carpeta processing/ aloja el pipeline de inferencia paso a paso, desde la extracción de pose hasta la generación de informes. Finalmente, entrenamiento/ contiene todos los scripts necesarios para reproducir el ciclo de entrenamiento de los modelos, junto con los datasets intermedios y los registros de experimentos. Los binarios entrenados listos para producción residen en modelos/vN/, donde N identifica la versión activa.

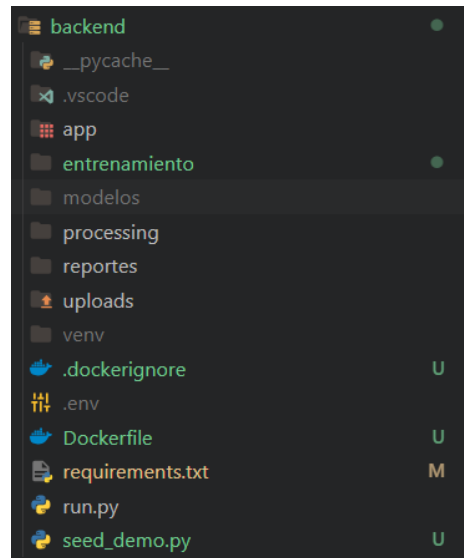


Figura 2. Carpeta Backend

Dentro de frontend/, el código fuente Android se organiza bajo el paquete `com.moveinsight` siguiendo estrictamente la separación en capas de Clean Architecture. La capa `core/` agrupa los componentes transversales (inyección de dependencias con Hilt, interceptores de red, almacenamiento seguro de tokens, WorkManager y utilidades compartidas). Las capas `data/` y `domain/` implementan el patrón repositorio con inversión de dependencias, de modo que la lógica de negocio nunca depende directamente de Retrofit ni de ninguna tecnología de infraestructura. La capa `presentation/` contiene los Composables, ViewModels y estados de UI organizados por funcionalidad (autenticación, captura, resultados, historial, bienestar y navegación).

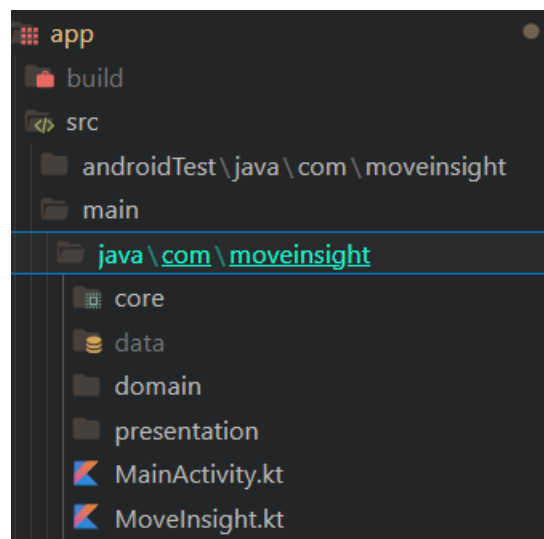


Figura 3. Carpeta frontend/app

En cuanto a las convenciones de nombrado, el proyecto sigue los estándares idiomáticos de cada ecosistema: `snake_case` para módulos, funciones y variables Python en el backend; `PascalCase` para clases Kotlin y `camelCase` para funciones y propiedades en el frontend. Los endpoints de la API siguen la convención `/api/v1/<recurso>`. Los modelos de IA se versionan con el prefijo `v` seguido de número entero (`v16` es la versión activa en el momento de redactar este manual), y su sustitución requiere únicamente actualizar la variable `MODEL_DIR` en `config.py` y reconstruir el contenedor de la API.

2 Backend

El backend de MoveInsight es una aplicación Python estructurada en capas que fluyen desde la configuración hasta el procesamiento de vídeo. La capa más externa es la API REST, que recibe peticiones del cliente Android, las valida y las autentica. Bajo ella se encuentran los servicios de lógica de negocio y la capa de acceso a datos, que persiste la información en MariaDB mediante el ORM SQLAlchemy. Por último, el pipeline de inteligencia artificial opera de forma asíncrona en segundo plano, sin bloquear las respuestas al cliente.

Toda la carpeta backend/app/ sigue esta separación: routers/ define los endpoints, services/ encapsula la lógica de negocio y models.py/database.py gestionan la persistencia. El procesamiento de vídeo está deliberadamente separado en processing/ para mantener el núcleo de la API independiente del código de IA.

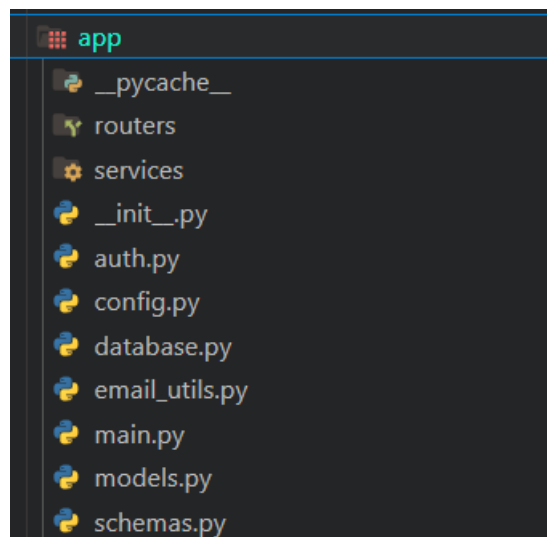


Figura 4. Carpeta backend/app

2.1 Arranque y configuración

El punto de entrada del servidor y su configuración centralizada determinan el comportamiento de toda la aplicación. Esta capa es la primera que se ejecuta al levantar el contenedor y la que hace disponibles al resto del código las variables de entorno, las rutas de ficheros y los parámetros de los modelos. Podemos visualizar la carpeta backend completa en la Figura 2. Carpeta Backend. Estos ficheros son los principales ficheros de arranque y configuración:

- **run.py:** Lanza Uvicorn con recarga automática en desarrollo. Es el único punto desde el que se arranca el servidor directamente fuera de Docker; en producción, el

contenedor invoca Uvicorn directamente con los parámetros definidos en el Dockerfile.

- **app/main.py:** Crea la instancia FastAPI, registra el ciclo de vida (lifespan) para tareas de arranque y apagado, configura CORS para permitir las peticiones del cliente Android, sirve los ficheros estáticos (informes PNG y vídeos con esqueleto) e inyecta todos los routers bajo el prefijo /api/v1.
- **app/config.py:** Clase Settings que hereda de BaseSettings de Pydantic y carga automáticamente el fichero .env. Centraliza todos los parámetros del sistema: DATABASE_URL, JWT_SECRET_KEY, JWT_ALGORITHM (HS256), JWT_EXPIRE_HOURS (1 h), JWT_REFRESH_EXPIRE_DAYS (30 días), EMAIL_HOST, EMAIL_PORT y MODEL_DIR, que apunta a la versión activa de los modelos (modelos/v16/). Cambiar la versión de modelo en producción implica únicamente modificar MODEL_DIR y reconstruir el contenedor.

2.2 Capa de datos: ORM y base de datos

La persistencia del sistema se gestiona a través de SQLAlchemy en modo ORM, lo que permite definir el esquema de la base de datos como clases Python y ejecutar consultas sin escribir SQL directamente. La base de datos subyacente es MariaDB 11, que se levanta como un servicio independiente en Docker.

Podemos observar los siguientes ficheros correspondientes a este punto, podemos verlos en la Figura 4. Carpeta backend/app:

- **app/database.py:** Crea el engine de SQLAlchemy a partir de DATABASE_URL, configura el pool de conexiones y expone SessionLocal (fábrica de sesiones) y la dependencia FastAPI get_db(), que inyecta una sesión de base de datos en cada endpoint y la cierra al finalizar la petición.
- **app/models.py:** Define las seis entidades ORM del sistema:
 - User: credenciales, perfil físico (peso, nivel, objetivo) y estado de verificación de correo.
 - EmailCode: códigos OTP de 6 dígitos para verificación de cuenta y reseteo de contraseña.
 - Session: cada subida de vídeo de un usuario. Contiene el estado de procesamiento (queued → processing → completed / failed), la ruta del vídeo, el peso y la puntuación de esfuerzo percibido (Borg).

- SessionResult: KPIs de cada repetición analizada (profundidad, torso, valgo, simetría y ritmo), almacenados como valores 0–10, donde los campos de valgo y simetría pueden ser nulos en sesiones de perfil lateral.
- Incident: incidencias físicas del usuario (zona corporal, severidad, tipo y fechas).
- PainCheckIn: registro de dolor a 24h/48h de cada sesión, con puntuación EVA y zona corporal.

2.3 DTOs y validación

Cada interacción con la API pasa por esquemas Pydantic definidos en `app/schemas.py`, que actúan como capa de validación entre el cliente y la lógica interna. FastAPI usa estos esquemas para deserializar el cuerpo de las peticiones, validar tipos y rangos automáticamente, y serializar las respuestas de salida.

Los esquemas están agrupados por dominio funcional: autenticación y registro de usuario, creación y consulta de sesiones, resultados de repeticiones, check-ins de dolor, incidencias, analytics, readiness y recomendaciones.

Cada entidad dispone habitualmente de al menos dos variantes: una de entrada (datos que envía el cliente) y una de salida (datos que devuelve la API), lo que permite exponer al cliente únicamente los campos necesarios sin filtrado manual en los routers.

Un ejemplo de un conjunto de clases almacenadas en el fichero:

```
class RefreshRequest(BaseModel):
    refresh_token: str

class MessageResponse(BaseModel):
    message: str

class VerifyEmailRequest(BaseModel):
    email: str
    code: str

class ResendVerificationRequest(BaseModel):
    email: str
```

2.4 Autenticación y seguridad

El sistema implementa autenticación basada en tokens JWT con doble token (acceso + refresco) y verificación de cuenta por correo electrónico. La lógica de seguridad está concentrada en dos módulos que son utilizados como dependencias por el resto de la aplicación. Lo podemos observar en los siguientes ficheros:

app/auth.py: Gestiona el ciclo completo de autenticación. Las contraseñas se almacenan hasheadas con bcrypt mediante passlib. Al hacer login, se generan dos tokens JWT firmados con HS256: un access token con expiración de 1 hora y un refresh token con expiración de 30 días. El endpoint de login rechaza expresamente los refresh tokens para evitar su uso como credenciales. La dependencia `get_current_user()` valida el token en cada petición protegida, comprueba que la cuenta esté verificada (`is_verified=True`) y devuelve el usuario activo. Todos los routers que requieren autenticación inyectan esta dependencia.

app/email_utils.py: Gestiona el envío de correos transaccionales a través de SMTP (Gmail). Genera códigos OTP de 6 dígitos para dos flujos: verificación de la cuenta al registrarse y reseteo de contraseña olvidada. Los códigos se persisten temporalmente en la tabla `EmailCode` con marca de tiempo para controlar su expiración.

2.5 Routers por dominio

Los routers son el punto de entrada de cada funcionalidad de la API. Cada fichero agrupa los endpoints de un dominio concreto, define las rutas, los métodos HTTP, los esquemas de entrada/salida y las dependencias (autenticación, base de datos). Todos están registrados bajo el prefijo `/api/v1/` en `main.py`.

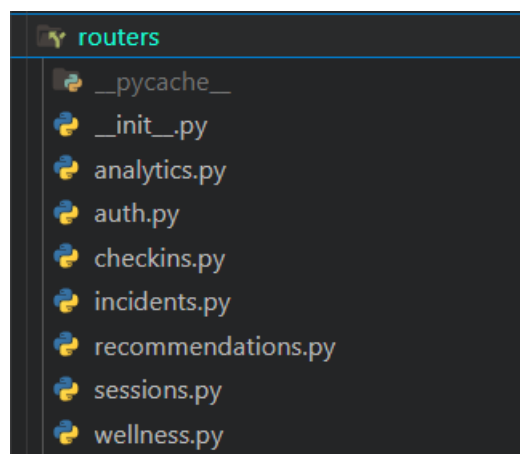


Figura 5. Carpeta backend/app/routers

- **auth.py:** Cubre el ciclo de vida de la cuenta de usuario: registro, verificación de correo, login, obtención de nuevo access token a partir del refresh token, cambio de contraseña con sesión activa, solicitud y confirmación de reseteo por correo, guardado de datos de onboarding (peso, nivel, objetivo) y eliminación de cuenta.
- **sessions.py:** Gestiona las sesiones de análisis. Recibe el vídeo por multipart/form-data junto con el peso y la puntuación Borg, persiste la sesión con estado queued y lanza el pipeline de IA como tarea en segundo plano devolviendo inmediatamente un 202 Accepted. Expone también el listado paginado de sesiones del usuario, el detalle de una sesión concreta con todos sus resultados por repetición y el vídeo con esqueleto superpuesto. Todas las consultas incluyen la comprobación `Session.user_id == current_user.id` para garantizar que cada usuario solo accede a sus propias sesiones.
- **analytics.py:** Proporciona los datos agregados del historial. Devuelve el resumen global del usuario (medias de KPIs, tendencias, récords), el desglose completo del semáforo de readiness con sus cuatro dimensiones, y la generación bajo demanda del informe PDF exportable.
- **checkins.py:** Registra y consulta los check-ins de dolor post-sesión. El cliente Android envía la puntuación EVA (0–10), las horas transcurridas desde la sesión (24h o 48h) y la zona corporal afectada. El router valida que el check-in corresponde a una sesión del propio usuario antes de persistirlo.
- **incidents.py:** CRUD completo de incidencias físicas. Permite registrar una incidencia con zona corporal, tipo, severidad y fechas de inicio y fin, consultar el historial, marcar incidencias como cerradas y eliminarlas. El campo `is_active` se calcula dinámicamente comparando la fecha de fin con la fecha actual.
- **wellness.py:** Ofrece la vista de bienestar longitudinal del usuario: el timeline retroactivo de readiness sesión a sesión, los factores de carga y dolor en el período previo a cada incidencia, y las validaciones que detectan patrones de sobrecarga o infracarga.
- **recommendations.py:** Evalúa el estado actual del usuario (KPIs, readiness, incidencias activas) contra un catálogo de triggers predefinidos y devuelve las recomendaciones pertinentes ordenadas por prioridad.

2.6 Servicios de lógica de negocio

Los servicios encapsulan la lógica de negocio más compleja, separándola de los routers para facilitar su prueba y mantenimiento independiente. Los routers delegan en ellos los cálculos que van más allá de la simple consulta a base de datos.

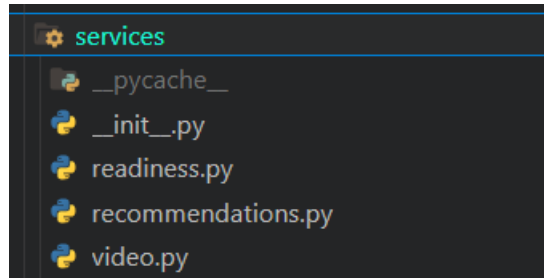


Figura 6. Carpeta backend/app/services

- **services/readiness.py:** Implementa el modelo de semáforo de readiness (0–100). Distingue tres etapas de madurez según el historial disponible: cold_start (menos de 3 sesiones), early (3–9 sesiones) y established (10 o más sesiones). El score se calcula como la ponderación de cuatro componentes: fatiga acumulada (con decaimiento exponencial de semivida 3 días), ratio de carga aguda/crónica ACWR con zona óptima 0,8–1,3, dolor post-sesión reciente (decaimiento de semivida 4 días) y calidad técnica media. Las ponderaciones de cada componente varían según la etapa de madurez.
- **services/recommendations.py:** Mantiene un catálogo estático de recomendaciones con texto, prioridad y condición de activación. En cada consulta evalúa los triggers sobre el estado actual del usuario (readiness bajo, KPI específico deficiente, incidencia activa, ACWR fuera de rango) y filtra las recomendaciones que aplican, devolviéndolas ordenadas por prioridad descendente.

2.7 Pipeline de IA en inferencia

El pipeline de inteligencia artificial es el núcleo técnico de MoveInsight. Toma un vídeo de sentadilla y produce, de forma completamente automática, la puntuación de cada KPI por repetición, un informe visual en PNG y un vídeo con el esqueleto superpuesto. Todo el procesamiento ocurre en segundo plano tras la subida del vídeo, sin bloquear al cliente.

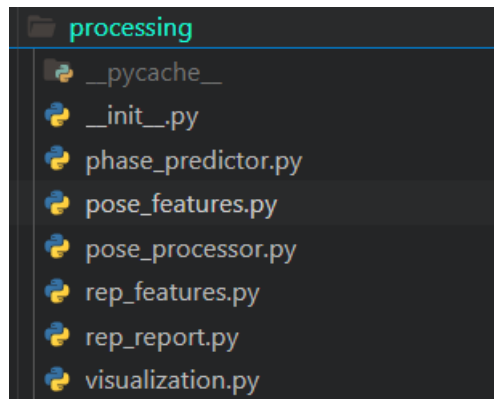


Figura 7. Carpeta backend/processing

- **app/services/video.py:** Organizador principal del pipeline. Es invocado por el router de sesiones como BackgroundTask y coordina todos los pasos: carga los modelos como singleton (para no recargarlos en cada sesión), actualiza el estado de la sesión en la BD a lo largo del proceso, llama secuencialmente a cada módulo de processing/ y persiste los resultados finales. Si cualquier paso falla, captura la excepción, marca la sesión como failed y registra el error en el mensaje de la sesión.
- **processing/pose_processor.py:** Aplica MediaPipe BlazePose sobre cada fotograma del vídeo y extrae los 33 landmarks del cuerpo humano con sus coordenadas normalizadas (x, y, z) y el score de visibilidad. Gestiona la apertura y lectura del fichero de vídeo fotograma a fotograma y devuelve la secuencia temporal de landmarks.
- **processing/pose_features.py:** Transforma la secuencia bruta de landmarks en características biomecánicas interpretables. Calcula ángulos articulares (rodilla, cadera, tobillo, columna) y distancias relevantes, aplica el filtro Savitzky-Golay para suavizar el ruido de MediaPipe y detecta automáticamente el perfil de grabación (frontal o lateral) a partir de la posición relativa de los landmarks de cadera y hombros. El perfil condiciona qué KPIs pueden calcularse: el valgo y la simetría solo están disponibles en perfil frontal.

- **processing/phase_predictor.py:** Clasifica cada fotograma en una de las cuatro fases de la sentadilla (de pie, descenso, fondo y ascenso) usando el modelo BiLSTM cargado desde `modelos/v16/bilstm_phases.keras`. Aplica las estadísticas de normalización almacenadas en `bilstm_phases.normstats.npz` antes de la inferencia. Una máquina de estados postprocesa las predicciones fotograma a fotograma para garantizar transiciones coherentes y extraer el conteo de repeticiones completas.
- **processing/rep_features.py:** Dado el listado de repeticiones detectadas, extrae un vector de 37 características por repetición que describe la calidad biomecánica de cada una: métricas de profundidad máxima, inclinación del torso, desviación lateral de rodillas, asimetría entre extremidades, duración y ritmo de cada fase, entre otras.
- **Clasificación de KPIs con XGBoost:** Los vectores de 37 características de cada repetición se pasan a los modelos XGBoost almacenados en `xgb_squat_models_frontal.pkl` y `xgb_squat_models_lateral.pkl`, seleccionando uno u otro según el perfil detectado. Hay un modelo independiente por cada KPI, y la clase predicha (1, 2 o 3) se mapea a un score continuo en la escala 0–10 que es lo que finalmente ve el usuario.
- **processing/rep_report.py:** Genera la imagen de informe en PNG con la evolución de los KPIs a lo largo de las repeticiones de la sesión. Esta imagen se guarda en `reportes/` y su ruta queda almacenada en `SessionResult`.
- **processing/visualization.py:** Renderiza el vídeo de esqueleto superpuesto: dibuja los landmarks y conexiones del cuerpo humano sobre cada fotograma y añade la anotación de la fase activa y el número de repetición. El vídeo resultante se guarda en `uploads/` y está disponible en el endpoint de esqueleto del router de sesiones.
- **Modelos/v16/:** Los binarios listos para producción son cuatro: `bilstm_phases.keras` (modelo de segmentación de fases), `bilstm_phases.normstats.npz` (media y desviación típica para normalizar las features de entrada), `xgb_squat_models_frontal.pkl` y `xgb_squat_models_lateral.pkl` (un estimador XGBoost por KPI para cada perfil). Todos se cargan una sola vez en arranque del servidor mediante el patrón singleton en `video.py`. En la carpeta `modelos` tenemos las diferentes versiones de los modelos que hemos tenido a lo largo de toda la aplicación siendo el más actual la versión 16 (v16) dentro de cada carpeta se encuentran los binarios de cada versión.

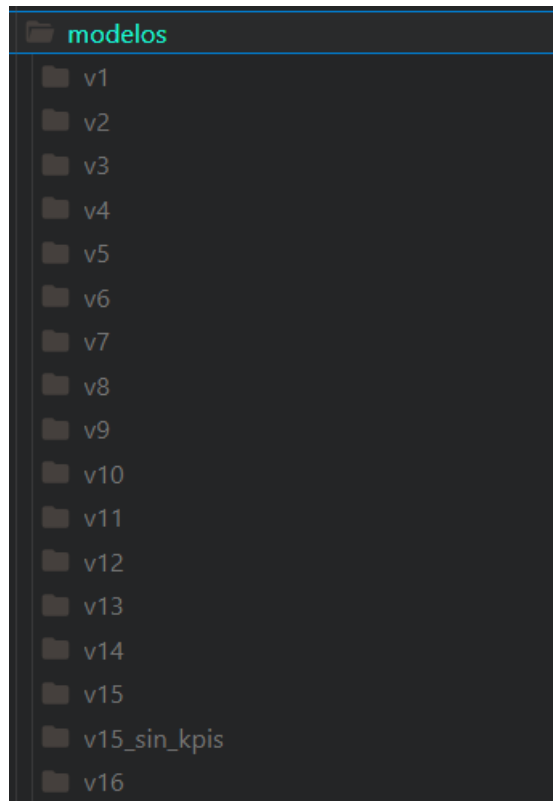


Figura 8. Carpeta backend/modelos

2.8 Contrato de la API REST y documentación OpenAPI

Todos los endpoints de la API siguen la convención de prefijo `/api/v1/` y utilizan JSON como formato de intercambio, a excepción de la subida de vídeos, que usa `multipart/form-data`. Las rutas identifican recursos en plural y kebab-case (`/sessions`, `/pain-checkins`, `/recommendations`). Los métodos HTTP se utilizan con su semántica estándar: GET para lecturas, POST para creaciones, PUT para actualizaciones completas y DELETE para eliminaciones.

Los códigos de respuesta más relevantes son: 200 OK para lecturas y actualizaciones correctas, 201 Created para recursos creados de forma síncrona, 202 Accepted para la subida de vídeo (procesamiento en segundo plano), 400 Bad Request para errores de validación, 401 Unauthorized para tokens inválidos o expirados, 403 Forbidden cuando el recurso existe pero no pertenece al usuario autenticado, 404 Not Found cuando el recurso no existe y 422 Unprocessable Entity para errores de esquema Pydantic. Los errores devuelven siempre un objeto JSON con el campo `detail` que describe el motivo, lo que permite al cliente Android mostrar mensajes informativos o reaccionar

programáticamente (por ejemplo, ante un 401 el TokenAuthenticator inicia el flujo de refresco automático).

FastAPI genera automáticamente la documentación interactiva de todos los endpoints a partir de los esquemas Pydantic y las anotaciones de los routers, manteniéndola siempre sincronizada con el código sin herramientas adicionales. Con el servidor en marcha está disponible en tres formatos:

- **/docs:** Interfaz interactiva que agrupa todos los endpoints por router y permite probarlos en vivo. Para cada uno muestra el método HTTP, la ruta, los parámetros de entrada con sus tipos y el esquema de respuesta con los posibles códigos de estado. Accesible en <https://prodiasv07.fis.usal.es/docs>.

Auth

POST	/api/v1/auth/register	Register
POST	/api/v1/auth/verify-email	Verify Email
POST	/api/v1/auth/resend-verification	Resend Verification
POST	/api/v1/auth/token	Login
POST	/api/v1/auth/refresh	Refresh Token
POST	/api/v1/auth/forgot-password	Forgot Password
POST	/api/v1/auth/reset-password	Reset Password
PUT	/api/v1/auth/change-password	Change Password
POST	/api/v1/auth/onboarding	Complete Onboarding
GET	/api/v1/auth/me	Get Me
DELETE	/api/v1/auth/me	Delete Account

Sessions

POST	/api/v1/sessions/upload	Upload Session
DELETE	/api/v1/sessions/history	Clear History
GET	/api/v1/sessions/{session_id}	Get Session

Figura 9. Documentación de la API /docs

- **/redoc:** Vista alternativa con diseño de dos columnas, más apropiada para lectura y captura formal. Accesible en <https://prodiasv07.fis.usal.es/redoc> .

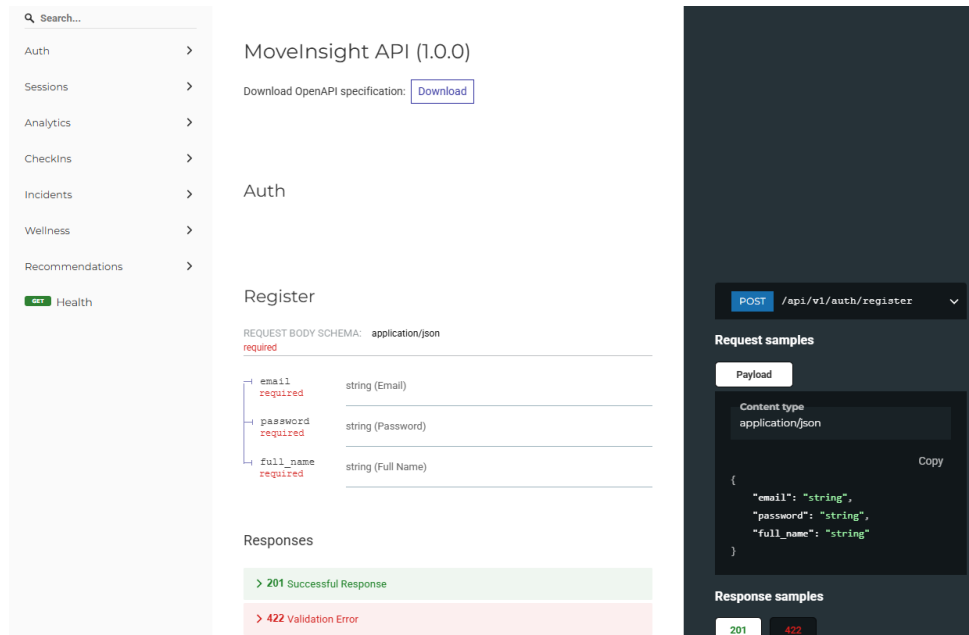


Figura 10. Documentación de la API /redoc

- **/openapi.json:** El contrato OpenAPI 3.0 completo en formato JSON, descargable y versionable junto al código para registrar la evolución del contrato entre versiones del sistema, disponible en: <https://prodiasv07.fis.usal.es/openapi.json> .

```
{
  "openapi": "3.1.0",
  "info": {
    "title": "MoveInsight API",
    "version": "1.0.0"
  },
  "paths": {
    "/api/v1/auth/register": {
      "post": {
        "tags": [
          "Auth"
        ],
        "summary": "Register",
        "operationId": "register_api_v1_auth_register_post",
        "requestBody": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/UserCreate"
              }
            },
            "required": true
          },
          "responses": {
            "201": {
              "description": "Successful Response",
              "content": {
                "application/json": {
                  "schema": {
                    "$ref": "#/components/schemas/UserResponse"
                  }
                }
              }
            },
            "422": {
              "description": "Validation Error",
              "content": {
                "application/json": {
                  "schema": {
                    "$ref": "#/components/schemas/HTTPValidationError"
                  }
                }
              }
            }
          }
        }
      }
    }
  }
}
```

Figura 11. Documentación de la API /openapi.json

3 Frontend

La aplicación Android de MoveInsight está desarrollada en Kotlin con Jetpack Compose y organizada siguiendo los principios de Clean Architecture en cuatro capas: core, data, domain y presentation. Esta separación garantiza que la lógica de negocio no dependa de ninguna tecnología concreta de UI o red, lo que facilita el mantenimiento, y la ampliación de funcionalidades.

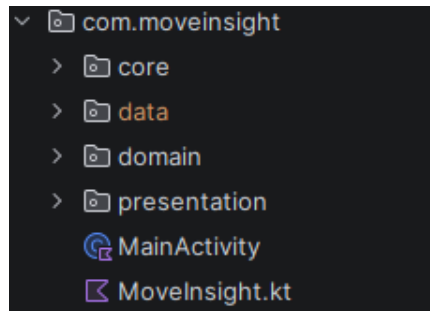


Figura 12. Carpeta frontend/src/com.moveinsight

3.1 Configuración Gradle

Los ficheros de construcción Gradle definen el objetivo, las dependencias, las variantes de compilación y las URLs de conexión al backend.

Son el punto de partida para cualquier desarrollador que incorpore el proyecto, ya que determinan qué versiones de las librerías están en uso y cómo se empaqueta el APK final.

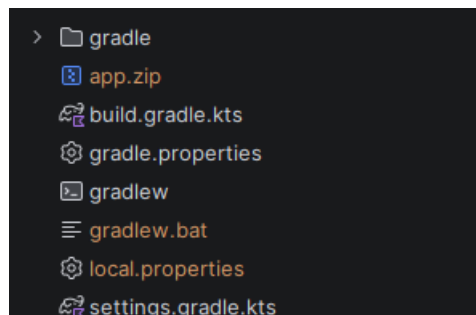


Figura 13. Carpeta frontend/src/gradle

- **settings.gradle.kts:** Declara el nombre del proyecto y los repositorios de dependencias (Maven Central, Google).
- **build.gradle.kts:** Configura los plugins globales de Android y Kotlin y las versiones del classpath de Hilt.

- **app/build.gradle.kts:** Fichero principal de compilación. Define minSdk, targetSdk y compileSdk, habilita Jetpack Compose y su compilador, y declara todas las dependencias: Retrofit, OkHttp, Hilt, Navigation Compose, CameraX, DataStore, WorkManager, Coil (carga de imágenes) y las librerías de gráficas. También define los buildTypes (debug y release) y los buildConfigField que inyectan la URL base del backend en tiempo de compilación, de forma que el APK de desarrollo apunta al servidor local y el de producción al servidor USAL.

3.2 Punto de entrada

El punto de entrada de la aplicación Android comprende dos clases que inicializan el grafo de Hilt y definen el *host* Compose raíz desde el que arranca toda la navegación. Estos ficheros se pueden observar en la Figura 12. Carpeta frontend/src/com.moveinsight

- **MoveInsight.kt:** Clase Application anotada con @HiltAndroidApp. Su única responsabilidad es inicializar el grafo de inyección de dependencias de Hilt y configurar WorkManager con la fábrica de workers de Hilt, necesaria para que los workers puedan recibir dependencias inyectadas.
- **MainActivity.kt:** Actividad única de la aplicación (patrón single-activity). Configura el tema Material3, establece el host de Compose con setContent y delega toda la lógica de navegación en NavGraph. También gestiona los deep links de las notificaciones de check-in de dolor, que abren directamente la pantalla de registro EVA cuando el usuario pulsa la notificación.

3.3 Capa core

La capa core agrupa todos los componentes transversales que son utilizados por el resto de la aplicación pero que no pertenecen a ningún dominio de negocio concreto. Contiene la configuración de red, el almacenamiento seguro de credenciales, el bus de eventos de sesión, el sistema de notificaciones y las utilidades compartidas.

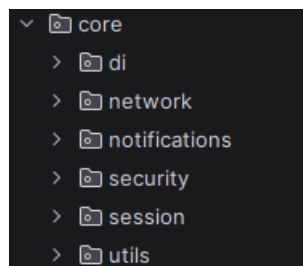


Figura 14. Carpeta frontend/src/com.moveinsight/core

- **core/di/:** Módulos Hilt que construyen y proveen las dependencias a toda la aplicación. NetworkModule instancia el cliente OkHttp con los interceptores y el adaptador Retrofit; RepositoryModule enlaza cada interfaz de repositorio con su implementación concreta; WorkerModule proporciona la fábrica de workers de Hilt.
- **core/network/AuthInterceptor:** Interceptor OkHttp que añade automáticamente la cabecera Authorization: Bearer <token> a todas las peticiones salientes, recuperando el access token del UserPreferencesDataStore.
- **core/network/TokenAuthenticator:** Autenticador OkHttp que se activa cuando el servidor devuelve un 401 Unauthorized. Solicita un nuevo access token usando el refresh token almacenado y reintenta la petición original de forma transparente. Si el refresco también falla (token caducado o revocado), emite un evento de cierre de sesión a través del SessionManager.
- **core/security/UserPreferencesDataStore:** Almacena de forma persistente y segura (Proto DataStore) el access token, el refresh token y los datos básicos del perfil del usuario (nombre, email, nivel). Es la única fuente de verdad de las credenciales en el cliente.
- **core/session/SessionManager:** Bus de eventos singleton implementado con un SharedFlow. Cuando el TokenAuthenticator detecta que la sesión ha expirado de forma irrecuperable, emite un evento que MainActivity recoge para redirigir al usuario a la pantalla de login y limpiar el estado de navegación.
- **core/notifications/:** Gestiona el sistema de notificaciones periódicas. PainCheckInWorker se ejecuta a las 24h y 48h después de cada sesión para recordar al usuario que registre su dolor post-entrenamiento; WeeklySummaryWorker genera un resumen semanal o una alerta de sobreentrenamiento los lunes. NotificationHelper centraliza la creación de canales y la construcción de notificaciones. SchedulePainNotificationsUseCase y ScheduleWeeklySummaryUseCase encapsulan la lógica de programación con WorkManager.
- **core/utills/:** Utilidades compartidas sin lógica de negocio: la clase sellada Resource<T> (Loading / Success / Error) que estandariza las respuestas de los repositorios, la función safeApiCall que envuelve las llamadas Retrofit en un bloque try-catch y mapea las excepciones a Resource.Error, Constants con valores globales y extensiones de Uri para preparar ficheros de vídeo como MultipartBody.

3.4 Capa data

La capa data implementa las interfaces de repositorio definidas en domain y es la única capa que conoce Retrofit, los DTOs de red y los detalles de serialización JSON. Su responsabilidad es transformar los datos remotos en modelos de dominio y encapsular los errores de red en `Resource<T>`.

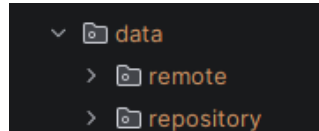


Figura 15. Carpeta `frontend/src/com.moveinsight/data`

- **data/remote/:** Interfaces Retrofit anotadas con `@GET`, `@POST`, `@PUT`, `@DELETE` y `@Multipart`. Hay una interfaz por dominio: `AuthService`, `SessionService`, `AnalyticsService`, `CheckInService`, `IncidentService`, `WellnessService` y `RecommendationService`. Los DTOs de respuesta son data classes Kotlin con anotaciones `@SerializedName` que mapean los nombres de campo JSON a nombres idiomáticos Kotlin.
- **data/repository/:** Implementaciones concretas de cada interfaz de repositorio. Cada implementación llama al servicio Retrofit correspondiente dentro de `safeApiCall`, convierte el DTO de respuesta al modelo de dominio mediante funciones de extensión `toModel()` y devuelve un `Resource<T>`. Esta capa también implementa los casos de uso que necesitan lógica adicional de combinación de datos, como `AnalyticsRepositoryImpl` y `PainCheckInRepositoryImpl`.

3.5 Capa domain

La capa domain es el núcleo de la aplicación y no contiene ninguna dependencia de Android, Retrofit ni ninguna otra tecnología externa. Define qué puede hacer la aplicación (casos de uso), con qué datos trabaja (modelos de dominio) y qué contratos deben cumplir las capas externas (interfaces de repositorio).

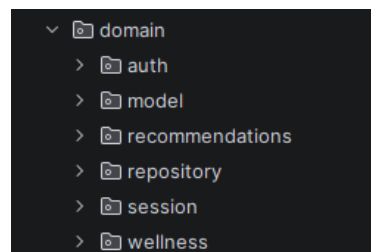


Figura 16. Carpeta `frontend/src/com.moveinsight/data`

- **domain/model/:** Modelos de negocio puros: User, Session, RepResult, Analytics, Incident, PainCheckIn, Recommendation, ReadinessBreakdown, WellnessFactor y otros. Son data classes Kotlin sin anotaciones de red ni de base de datos, lo que las hace completamente independientes de la infraestructura.
- **domain/repository/:** Interfaces que definen el contrato que debe cumplir la capa data. La capa domain depende únicamente de estas interfaces, nunca de las implementaciones concretas, lo que permite sustituir el backend de red por cualquier otra fuente de datos sin tocar la lógica de negocio.
- **Casos de uso por dominio:** Cada caso de uso encapsula una única operación de negocio y recibe sus dependencias por constructor mediante Hilt. Los once casos de uso de autenticación (domain/auth/) cubren registro, verificación, login, refresco, cambio de contraseña y eliminación de cuenta. Los de sesión (domain/session/) gestionan la subida, el listado y el polling de estado. Los de bienestar (domain/wellness/) registran incidencias y check-ins de dolor. Los casos de uso especiales `GetReadinessBreakdownUseCase`, `GetAnalyticsUseCase`, `ExportReportUseCase` y `SubmitPainCheckInUseCase` combinan llamadas a varios repositorios o añaden transformaciones de datos antes de exponer el resultado al ViewModel.

3.6 Capa presentation

La capa de presentación contiene los Composables y ViewModels organizados por funcionalidad. Cada pantalla sigue el mismo patrón: el ViewModel expone un UiState inmutable mediante StateFlow y los eventos de efecto único (navegación, snackbars) mediante Channel; el Composable observa el estado con `collectAsStateWithLifecycle` y envía acciones al ViewModel.

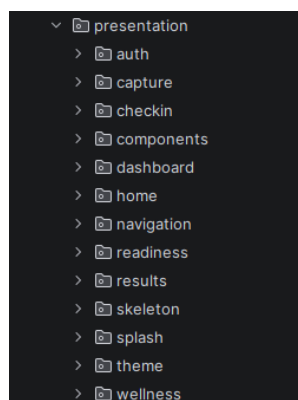


Figura 17. Carpeta frontend/src/com.moveinsight/presentation

- **presentation/auth/:** Pantallas del ciclo de autenticación: SplashScreen (comprueba si hay token válido y redirige), LoginScreen, RegisterScreen (formulario de registro con validación en tiempo real), VerifyEmailScreen (introducción del código OTP de 6 dígitos), ForgotPasswordScreen, ResetPasswordScreen, ChangePasswordScreen (con sesión activa) y OnboardingScreen (recogida de peso, nivel y objetivo tras el primer login).
- **presentation/capture/:** CaptureScreen gestiona la grabación del vídeo con CameraX, la selección de vídeo desde galería y el formulario de metadatos (peso y puntuación Borg). BorgScaleDialog muestra la escala de esfuerzo percibido de Borg (6–20) con descripciones para que el usuario pueda estimarla correctamente.
- **presentation/results/:** ResultsScreen muestra los KPIs de cada repetición una vez el análisis ha completado. Durante el procesamiento muestra un indicador de carga con consejos rotativos (LoadingWithTips). También calcula y muestra el indicador de fatiga comparando el rendimiento con la sesión anterior e identifica si hay signos de sobreentrenamiento.
- **presentation/skeleton/:** SkeletonVideoScreen reproduce el vídeo con el esqueleto superpuesto generado por el backend, con controles de reproducción básicos.
- **presentation/home/:** HomeScreen es la pantalla principal de la aplicación. Muestra el semáforo de readiness con su valor actual y etiqueta de estado, las recomendaciones activas ordenadas por prioridad y el acceso rápido a la nueva sesión. HomeViewModel coordina la carga paralela de readiness y recomendaciones.
- **presentation/dashboard/:** DashboardScreen es el historial del usuario. ChartSection renderiza la evolución temporal de cada KPI mediante gráficas de líneas. SessionCard muestra el resumen de cada sesión en el listado. InsightCard destaca los récords personales del usuario (mejor sesión, mejor repetición por KPI).
- **presentation/readiness/:** ReadinessBreakdownScreen desglosa el semáforo de readiness en sus cuatro dimensiones (fatiga, ACWR, dolor y técnica), mostrando el valor y la contribución de cada una al score global, así como la etapa de madurez del modelo (cold_start, early o established).
- **presentation/checkin/:** PainCheckInScreen permite registrar el dolor post-sesión. BodyMapComponent muestra un mapa corporal interactivo donde el usuario selecciona la zona afectada. EvaScaleSlider es un control deslizante calibrado con la escala EVA (0–10) con etiquetas descriptivas en los extremos.

- **presentation/wellness/:** WellnessScreen muestra el timeline de bienestar con la evolución del readiness y las incidencias marcadas como eventos sobre la línea de tiempo. WellnessChart renderiza la gráfica de carga a lo largo del tiempo. IncidentFactorsScreen detalla los factores de carga y dolor en el período previo a una incidencia concreta. RegisterIncidentDialog permite registrar una nueva incidencia con zona, tipo, severidad y fechas.

3.7 Navegación y tema

La navegación y el sistema de diseño son componentes de la capa de presentación que afectan a todas las pantallas. La navegación centraliza el control de flujo de la aplicación y el tema garantiza la coherencia visual.

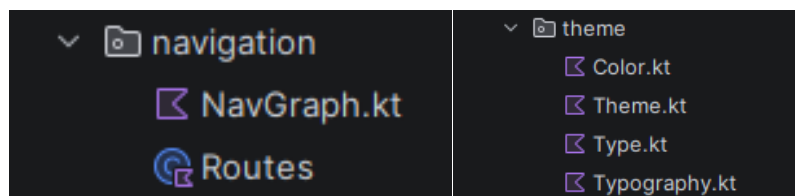


Figura 18. Carpeta frontend/src/com.moveinsight/presentation/navigation

- **navigation/Routes:** Objeto que define todas las rutas de la aplicación como constantes de cadena, incluyendo las rutas con parámetros (como el identificador de sesión en los resultados). El uso de constantes evita errores tipográficos en las llamadas de navegación.
- **navigation/NavGraph:** Composable raíz que configura el NavHost con todas las rutas y sus Composables de destino. Gestiona dos grafos anidados: el no autenticado (splash, login, registro y recuperación de contraseña) y el autenticado (resto de pantallas). Observa el SessionManager para redirigir al grafo no autenticado si la sesión expira en cualquier pantalla, limpiando el back stack para que el usuario no pueda volver atrás con el botón retroceso.
- **theme/Color, theme/Theme, theme/Type, theme/Typography:** Definen la paleta de colores, el tema Material3 (claro y oscuro) y la tipografía de la aplicación.
- **components/:** Componentes Compose reutilizables: MoveInsightButton (botón primario con estado de carga integrado), MoveInsightTextField (campo de texto con validación visual), MoveInsightLogo (logotipo de la aplicación), RecommendationCard (tarjeta de recomendación con indicador de prioridad por color) y LoadingWithTips (pantalla de espera con consejos biomecánicos rotativos).

4 Entrenamiento

El sistema de inteligencia artificial de MoveInsight se basa en dos modelos complementarios: un BiLSTM que segmenta el vídeo en fases de la sentadilla y un conjunto de modelos XGBoost que clasifican la calidad biomecánica de cada repetición. Ambos son el resultado de un ciclo de entrenamiento supervisado que parte de vídeos reales etiquetados manualmente y termina en binarios listos para producción, versionados y evaluados de forma reproducible.

El ciclo completo sigue esta secuencia: etiquetado manual de vídeos → extracción de features frame a frame → construcción de datasets → augmentación por espejo → entrenamiento → evaluación → versionado → despliegue.

Podemos seguir entrenando el modelo sin la necesidad de hacer cambios en el sistema, esto es vital para poder seguir mejorando la app. Una vez reentrenado un modelo simplemente habrá que cambiar una variable global (para introducir el nombre de la nueva versión del modelo) y añadir el modelo nuevo a la carpeta de modelos. Se explica con más detalle en el último punto.

Todos los scripts necesarios para reproducir este ciclo residen en backend/entrenamiento/.

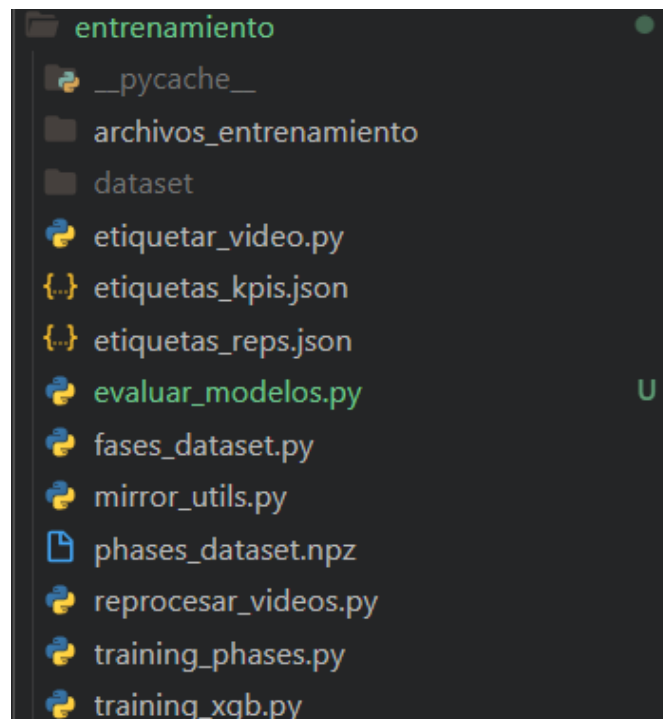


Figura 19. Carpeta backend/entrenamiento

4.1 Preparación de datos: etiquetado, *features* y dataset

Antes de entrenar cualquier modelo es necesario disponer de datos etiquetados de calidad. Esta fase transforma los vídeos brutos en estructuras numéricas listas para el entrenamiento, pasando por el etiquetado manual, la extracción de características y la augmentación del dataset.

- **etiquetar_video.py:** Herramienta interactiva basada en OpenCV que permite al etiquetador marcar fotograma a fotograma el inicio y fin de cada repetición y de cada fase (de pie, descenso, fondo y ascenso). El resultado se guarda en `etiquetas_reps.json`, con la estructura `{video_id: [{start, end, phases: [...]}]}`.
- **etiquetas_kpis.json:** Fichero de etiquetas de calidad por repetición. Para cada vídeo y cada repetición registra la puntuación de cada KPI en una escala 1–3 (malo, aceptable, bueno), el perfil de grabación (frontal o lateral) y los KPIs no medibles en ese perfil (valgo y simetría en lateral). Estas etiquetas alimentan el entrenamiento de XGBoost.
- **reprocesar_videos.py:** Script de procesamiento en lote que aplica el módulo `processing/pose_processor.py` sobre todos los vídeos del conjunto de entrenamiento y guarda las secuencias de landmarks en ficheros `.npy` con forma `(T, 33, 3)`, donde `T` es el número de fotogramas. Permite regenerar los ficheros intermedios si se actualiza la versión de MediaPipe.
- **fases_dataset.py:** Combina los `.npy` de landmarks con las etiquetas de `etiquetas_reps.json` para construir el dataset de segmentación de fases. Aplica la extracción de features biomecánicas (`processing/pose_features.py`), asigna la etiqueta de fase a cada fotograma dentro de cada repetición etiquetada y genera el fichero `phases_dataset.npz` con los arrays `X` (features) e `y` (clases 0–3) listos para alimentar el BiLSTM.
- **mirror_utils.py:** Módulo de augmentación por espejo horizontal. Replica cada secuencia invirtiéndola lateralmente: intercambia los landmarks izquierda-derecha, niega los valores de valgo (que invierten su signo al reflejar el plano) y mantiene el resto de features inalteradas. Esta técnica duplica el tamaño del dataset de forma coherente con la física del movimiento, balanceando los perfiles grabados desde distintos lados.

4.2 Entrenamiento de modelos

Con los datasets preparados y aumentados, se entrenan los dos modelos que componen el pipeline de inferencia. Cada uno tiene su propio script de entrenamiento con configuración de hiperparámetros, estrategia de validación y exportación del binario final.

- **training_phases.py:** Entrena el BiLSTM de segmentación de fases. Realiza la partición por vídeo (no por fotograma) para evitar fuga de datos entre entrenamiento y validación. Normaliza las features calculando media y desviación típica sobre el conjunto de entrenamiento y las guarda en `bilstm_phases.normstats.npz` para que la inferencia aplique exactamente la misma transformación. La arquitectura BiLSTM incluye una capa de Masking que maneja secuencias de distinta longitud mediante padding. Aplica pesos de clase para compensar el desbalanceo entre fases (la fase de fondo dura menos fotogramas que el ascenso o descenso) y usa callbacks de early stopping y reducción de learning rate para evitar sobreentrenamiento. El modelo final se exporta como `bilstm_phases.keras`.
- **training_xgb.py:** Entrena un modelo XGBoost independiente por cada KPI (profundidad, torso, valgo, simetría y ritmo) y por cada perfil (frontal y lateral), resultando en dos grupos de pickles: `xgb_squat_models_frontal.pkl` y `xgb_squat_models_lateral.pkl`. Utiliza el vector de 37 features por repetición generado por `processing/rep_features.py`. Para cada modelo aplica validación cruzada estratificada, oversampling de clases minoritarias con SMOTE y tuning automático de hiperparámetros con Optuna. Las propuestas de análisis intermedias de Optuna se guardan en `propuestas_analisis/` para auditoría.

4.3 Evaluación, versionado y producción

Una vez entrenados los modelos, el ciclo concluye con su evaluación objetiva, su versionado y su despliegue en el servidor. Este proceso garantiza que cada versión de los modelos sea reproducible y que las regresiones se detecten antes de actualizar producción.

- **backend/reportes/:** permite comparar versiones a lo largo del tiempo y detectar regresiones al actualizar MediaPipe o ampliar el dataset.
- **Estructura de versionado (modelos/vN/):** Cada versión entrenada se almacena en su propia carpeta numerada (`v1/`, `v2/`, ..., `v16/`). El contenido de cada carpeta es siempre el mismo conjunto de cuatro artefactos: `bilstm_phases.keras`, `bilstm_phases.normstats.npz`, `xgb_squat_models_frontal.pkl` y

xgb_squat_models_lateral.pkl. La variable MODEL_DIR en backend/app/config.py apunta a la versión activa. Cambiar de versión en producción implica únicamente actualizar ese valor y reconstruir el contenedor de la API (docker compose up --build api), sin modificar ningún código del pipeline de inferencia. Para hacer rollback basta con apuntar MODEL_DIR a la versión anterior.

Glosario de términos

Se recogen a continuación los términos técnicos y específicos del proyecto empleados a lo largo de este anexo, con el fin de facilitar su comprensión.

- **202 Accepted.** Código de respuesta HTTP que devuelve la API al subir un vídeo: la petición se ha aceptado y el análisis se ejecuta en segundo plano, sin esperar a que termine.
- **ACWR (Acute:Chronic Workload Ratio).** Cociente entre la carga de entrenamiento aguda (últimos 7 días) y la crónica (media de las últimas 4 semanas); es una de las cuatro dimensiones del readiness.
- **APK (Android Package Kit).** Fichero empaquetado e instalable de la aplicación Android que produce Gradle.
- **ASGI (Asynchronous Server Gateway Interface).** Interfaz estándar entre un servidor web y una aplicación Python asíncrona; FastAPI la implementa y se sirve mediante Uvicorn.
- **AuthInterceptor.** Interceptor de OkHttp que añade automáticamente la cabecera Authorization: Bearer <token> con el access token a cada petición saliente.
- **Augmentación por espejo (data augmentation).** Técnica que duplica el dataset reflejando lateralmente cada secuencia (intercambio de landmarks izquierda-derecha e inversión del signo del valgo), de forma coherente con la física del movimiento.
- **BackgroundTask.** Mecanismo de FastAPI para ejecutar una tarea (aquí, el pipeline de IA) después de responder al cliente, sin bloquear la respuesta.
- **BaseSettings / BaseModel.** Clases base de Pydantic: BaseSettings carga la configuración desde el fichero .env (clase Settings) y BaseModel define los esquemas de validación de datos.
- **bcrypt.** Función de hash con la que se almacenan las contraseñas (a través de passlib), resistente a ataques de fuerza bruta.
- **BiLSTM (Bidirectional Long Short-Term Memory).** Red neuronal recurrente bidireccional empleada para segmentar el vídeo en las fases de la sentadilla.
- **Borg (escala de).** Escala de esfuerzo percibido que el usuario indica al subir cada sesión; alimenta el cálculo de la carga de entrenamiento.

- **buildConfigField.** Campo de Gradle que inyecta valores (como la URL base del backend) en el APK en tiempo de compilación, con distinto valor para debug y release.
- **CameraX.** Biblioteca de Android empleada para la grabación de vídeo desde la pantalla de captura.
- **Caso de uso (use case).** Clase de la capa *domain* que encapsula una única operación de negocio y recibe sus dependencias por constructor mediante Hilt.
- **Clase sellada (sealed class).** Tipo de Kotlin con un conjunto cerrado de subtipos; se usa en `Resource<T>` para representar los estados Loading, Success y Error.
- **Clean Architecture.** Organización del código del cliente en capas (core, data, domain y presentation) en la que la lógica de negocio no depende de la tecnología de UI ni de red.
- **Coil.** Biblioteca de Android para la carga y el cacheo de imágenes.
- **Composable.** Función de Jetpack Compose que describe de forma declarativa un fragmento de la interfaz.
- **Contenedor (Docker).** Unidad aislada que empaqueta una pieza del sistema y sus dependencias; cambiar la versión de modelos implica reconstruir el contenedor de la API.
- **Convenciones de nombrado.** Estándares idiomáticos del proyecto: kebab-case en las rutas de la API, snake_case en Python, PascalCase en clases Kotlin y camelCase en funciones y propiedades Kotlin.
- **CORS (Cross-Origin Resource Sharing).** Mecanismo configurado en FastAPI que autoriza al cliente Android a consumir la API.
- **DataStore (Proto DataStore).** Almacenamiento local persistente y seguro de Android donde el cliente guarda los tokens y los datos básicos del perfil (UserPreferencesDataStore).
- **Deep link.** Enlace que abre directamente una pantalla concreta de la aplicación; aquí, la notificación de check-in abre la pantalla de registro de dolor (EVA).
- **Dependencia (FastAPI).** Función inyectable que FastAPI resuelve en cada petición, como `get_db()` (sesión de base de datos) o `get_current_user()` (usuario autenticado).
- **DTO / esquema Pydantic.** Objeto de transferencia de datos; en el backend, los esquemas Pydantic que validan la entrada y dan forma a la salida de cada endpoint, con variantes de entrada y de salida.

- **Early stopping (parada temprana).** Callback de entrenamiento que detiene el proceso cuando el rendimiento en validación deja de mejorar, evitando el sobreajuste.
- **Endpoint.** Punto de acceso de la API, identificado por una ruta y un método HTTP, que expone una operación.
- **Engine (SQLAlchemy).** Objeto que gestiona la conexión con la base de datos a partir de DATABASE_URL y del pool de conexiones.
- **EVA (Escala Visual Analógica).** Escala de 0 a 10 con la que el usuario registra el dolor en los check-ins a las 24 y 48 horas.
- **FastAPI.** Framework web de Python sobre el que se construye la API REST; genera automáticamente la documentación OpenAPI.
- **Feature (característica).** Variable biomecánica derivada de los landmarks (ángulos, distancias, velocidades); cada repetición se describe con un vector de 37 features.
- **Filtro de Savitzky-Golay.** Filtro de suavizado que atenúa el ruido de la estimación de pose ajustando un polinomio local, sin deformar los picos reales del movimiento.
- **Flujo reactivo (Flow / StateFlow / SharedFlow).** Flujos de Kotlin para programación reactiva: StateFlow expone el estado de la UI (UiState) y SharedFlow implementa el bus de eventos de sesión (SessionManager).
- **Fuga de datos (data leakage).** Contaminación que se produciría si repeticiones de un mismo vídeo cayeran a la vez en entrenamiento y validación; se evita particionando por vídeo.
- **Gradle.** Sistema de construcción del proyecto Android; los ficheros build.gradle.kts definen dependencias, variantes de compilación y empaquetado.
- **Gson.** Biblioteca de serialización JSON; las anotaciones @SerializedName mapean los campos JSON a las propiedades Kotlin de los DTOs.
- **Hilt.** Framework de inyección de dependencias de Android que construye el grafo de dependencias de la aplicación (NetworkModule, RepositoryModule, WorkerModule).
- **Hiperparámetro.** Parámetro de configuración del entrenamiento (no aprendido); en XGBoost se ajusta automáticamente con Optuna.
- **HS256.** Algoritmo de firma simétrica con el que se firman los tokens JWT.

- **Interceptor (OkHttp).** Componente que intercepta peticiones o respuestas de red para modificarlas; AuthInterceptor añade la cabecera de autorización.
- **Inversión de dependencias.** Principio por el que la capa *domain* define las interfaces de repositorio y la capa *data* las implementa, de modo que la lógica de negocio no depende de la infraestructura.
- **Jetpack Compose.** Toolkit declarativo de interfaz de Android con el que está construida toda la UI del cliente.
- **JWT (JSON Web Token).** Token firmado que acredita la identidad del usuario; el sistema emplea un access token (1 h) y un refresh token (30 días).
- **Keras (.keras).** Formato en el que se exporta el modelo BiLSTM entrenado (bilstm_phases.keras).
- **KPI (Key Performance Indicator).** Cada indicador biomecánico evaluado por repetición: profundidad, torso, valgo, simetría y ritmo.
- **Landmark.** Punto anatómico (articulación o referencia) detectado por MediaPipe BlazePose; se extraen 33 por fotograma con coordenadas (x, y, z) y un valor de visibilidad.
- **lifespan.** Gestor del ciclo de vida de FastAPI que ejecuta tareas al arrancar y apagar la aplicación.
- **MariaDB.** Sistema gestor de base de datos relacional sobre el que persiste el sistema, ejecutado como contenedor Docker.
- **Masking / padding.** Mecanismo de la red BiLSTM para manejar secuencias de distinta longitud: el *padding* las iguala y la capa de *Masking* ignora el relleno.
- **Material 3.** Sistema de diseño de Android sobre el que se define el tema (claro y oscuro) de la aplicación.
- **MediaPipe BlazePose.** Modelo de estimación de pose tridimensional que extrae los 33 landmarks del cuerpo a partir del vídeo.
- **MODEL_DIR.** Variable de configuración (config.py) que apunta a la carpeta de la versión de modelos activa (modelos/v16/); cambiar de versión consiste en actualizarla y reconstruir el contenedor.
- **multipart/form-data.** Formato de petición HTTP empleado para la subida del vídeo, frente a JSON en el resto de endpoints.
- **MVVM (Model-View-ViewModel).** Patrón de la capa de presentación: el ViewModel expone el estado y la lógica, y el Composable lo observa.

- **Navegación Compose (NavHost / NavGraph).** Componentes de navegación: NavHost aloja las rutas y NavGraph configura los grafos autenticado y no autenticado, gestionando el *back stack*.
- **NumPy (.npy / .npz).** Formatos de NumPy: .npy guarda las secuencias de landmarks con forma (T, 33, 3) y .npz agrupa varios arrays (datasets y estadísticas de normalización).
- **OkHttp.** Cliente HTTP de Android sobre el que opera Retrofit y en el que se instalan los interceptores y el autenticador.
- **OpenAPI.** Estándar de descripción de APIs REST; FastAPI lo genera automáticamente y lo expone en /docs, /redoc y /openapi.json.
- **Optuna.** Biblioteca de optimización con la que se ajustan automáticamente los hiperparámetros de los modelos XGBoost.
- **ORM (Object-Relational Mapping).** Técnica que permite definir el esquema y consultar la base de datos como clases Python; aquí mediante SQLAlchemy.
- **OTP (One-Time Password).** Código de un solo uso de 6 dígitos enviado por correo para verificar la cuenta o restablecer la contraseña (tabla EmailCode).
- **Oversampling / SMOTE.** Técnica para compensar el desbalanceo de clases generando ejemplos sintéticos de las clases minoritarias en el entrenamiento de XGBoost.
- **passlib.** Biblioteca de Python con la que se aplica el hash bcrypt a las contraseñas.
- **Perfil de grabación (frontal / lateral).** Plano desde el que se graba la sentadilla, detectado automáticamente; condiciona qué KPIs pueden medirse (el valgo y la simetría solo en frontal).
- **pickle (.pkl).** Formato de serialización de Python en el que se guardan los modelos XGBoost (xgb_squat_models_frontal.pkl y _lateral.pkl).
- **Pipeline de inferencia.** Secuencia de etapas del backend que, a partir del vídeo, extrae la pose, calcula las features, segmenta las repeticiones y clasifica los KPIs.
- **Polling (sondeo).** Estrategia del cliente que consulta periódicamente el estado de la sesión hasta que el análisis pasa a completed.
- **Pool de conexiones.** Conjunto de conexiones reutilizables a la base de datos que gestiona el engine de SQLAlchemy.
- **Pydantic.** Biblioteca de validación y serialización de datos de Python; define los esquemas (DTOs) y la configuración (BaseSettings).

- **Readiness.** Indicador global de disposición para el entrenamiento (0–100), mostrado como un semáforo, que combina fatiga, ACWR, dolor y técnica reciente.
- **Repositorio (patrón).** Abstracción de acceso a datos: la interfaz se define en *domain* y la implementación concreta, que conoce Retrofit, reside en *data*.
- **Resource<T>.** Clase sellada de Kotlin que estandariza las respuestas de los repositorios en tres estados: Loading, Success y Error.
- **Retrofit.** Cliente HTTP de alto nivel de Android con el que el frontend consume la API REST; las interfaces se anotan con @GET, @POST, etc.
- **Router.** Módulo de FastAPI que agrupa los endpoints de un dominio (auth, sessions, analytics...), todos bajo el prefijo /api/v1/.
- **safeApiCall.** Función de utilidad que envuelve las llamadas Retrofit en un try-catch y mapea las excepciones a Resource.Error.
- **@SerializedName.** Anotación de Gson que asocia el nombre de un campo JSON con la propiedad Kotlin correspondiente del DTO.
- **SessionManager.** Bus de eventos *singleton* (SharedFlow) que notifica el cierre de sesión cuando el refresco de token falla de forma irrecuperable.
- **Single-activity.** Patrón de Android en el que la aplicación tiene una única Activity (MainActivity) y toda la navegación ocurre con Compose.
- **SQLAlchemy.** ORM de Python con el que se define el esquema y se consulta MariaDB sin escribir SQL directamente.
- **UiState.** Estado inmutable de una pantalla que el ViewModel expone mediante StateFlow y que el Composable observa.
- **Uvicorn.** Servidor ASGI que ejecuta la aplicación FastAPI.
- **Validación cruzada agrupada (StratifiedGroupKFold).** Validación cruzada que mantiene juntas en el mismo pliegue todas las repeticiones de un mismo vídeo, evitando la fuga de datos.
- **ViewModel.** Componente de la capa de presentación que expone el estado de la UI (UiState) mediante StateFlow y procesa los eventos de la interfaz.
- **WorkManager.** Componente de Android para tareas en segundo plano fiables; programa las notificaciones de check-in de dolor y el resumen semanal.
- **XGBoost (Extreme Gradient Boosting).** Modelo de *gradient boosting* sobre árboles; un modelo independiente por cada KPI y perfil clasifica la calidad de cada repetición.